

ONLINE JUDGE

Problem Statement:

An Online Judge (OJ) is a web-based platform where programmers solve coding problems and submit their solutions. The submitted programs are automatically compiled, executed against hidden test cases, and evaluated. The platform generates a verdict such as Accepted, Wrong Answer, Time Limit Exceeded, or Runtime Error.

The system should support secure code execution, maintain user submissions, provide leaderboards, and handle multiple concurrent users without affecting system performance.

Project Overview:

The objective is to develop a scalable Online Judge using the MERN stack that allows users to:

- Register and login securely.
- Browse programming problems.
- Submit solutions in multiple programming languages.
- Execute code inside Docker containers.
- Evaluate submissions automatically.

The architecture follows a client-server model where the frontend communicates with REST APIs, and the backend manages authentication, problem management, code execution, and database operations.

Features:

The proposed Online Judge platform is designed to provide a complete environment for coding practice and programming contests. It enables users to register securely, log in to their accounts, and manage their personal profiles while maintaining a history of their submissions and achievements.

The platform provides a centralized repository of coding problems where users can browse, search, and filter problems based on difficulty levels. Each problem

contains a detailed problem statement, constraints, sample inputs, sample outputs, and hidden test cases used for evaluation.

To ensure system security and fairness, every submission is executed inside an isolated Docker container with predefined CPU, memory, and execution time limits. This sandboxed environment prevents malicious code from affecting the host system while ensuring consistent execution across all submissions.

Challenges:

- Users can submit programs containing malicious or unsafe code that may attempt to consume excessive system resources or access sensitive files.
- Sensitive information such as user credentials, hidden test cases, and submission results must be protected from unauthorized access.
- During coding contests, thousands of users may submit solutions simultaneously, causing excessive load on the server.
- As the number of users grows, a single server may not be sufficient to process all submissions efficiently.

Proposed Solution:

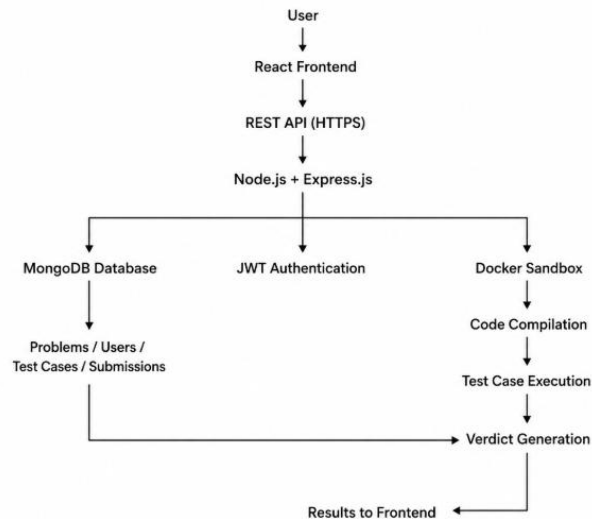
- **Docker-Based Code Sandboxing:** Every submitted program is executed inside an isolated Docker container with predefined CPU, memory, and execution time limits.
- **Secure Authentication and Authorization:** User authentication is implemented using JWT tokens, while passwords are securely stored using bcrypt hashing.
- **Asynchronous Submission Processing:** To handle thousands of simultaneous submissions, the system uses a message queue where each code submission is stored as a job.
- **Scalable Worker Architecture:** The evaluation system is designed using independent worker services that can be scaled horizontally.

Technology Stack:

- **Frontend:** React.js
- **Backend:** Node.js + Express.js
- **Database:** MongoDB

- **Authentication:** JWT + bcrypt
- **Sandbox Environment:** Docker
- **Testing:** Postman
- **API Architecture:** RESTful APIs

High-Level Architecture Diagram



HIGH LEVEL DESIGN:

1. Database Design

- **Collection 1: Users**

userId: ObjectId (Primary Key)

name: string

password: string

email: string

phone: number

- **Collection 2: Problems**

problemId: ObjectId (Primary Key)

statement: string

title: string

code: string

difficulty: string

constraints: string

- **Collection 3: Test Cases**

testCaseId: ObjectId (Primary Key)

input: string

output: string

problemId: (Foreign Key)

- **Collection 4: Submission**

submissionId: ObjectId (Primary Key)

problemId: (Foreign Key)

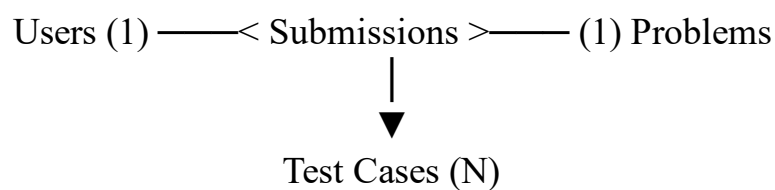
code: string

verdict: string

runtime: number

memory: number

submittedAt: date



2. Web Server Design

- **Screen 1: Home Page/ Dashboard**

Components: Login/Register, Problem List, Search Problems, Filter

- **Screen 2: Problem Page**

Components: Problem statement, Code editor, Language selection, Submit button, Verdict log

- **Screen 3: Leaderboard**

Components: Rank, Username, Submission Time

Module	Client Action	Server Action	API Endpoint
Problem Listing	Displays all available coding problems on the dashboard. Each problem title is clickable and redirects the user to its respective problem page.	Retrieves all coding problems from the Problems collection in MongoDB and returns them to the client in JSON format.	GET /api/problems
Individual Problem	Displays the complete problem statement, constraints, sample input/output, programming language selector, integrated code editor, and submit button.	Retrieves the selected problem using the problem ID from the Problems collection and returns the complete problem details.	GET /api/problems/:id
Code Submission	Sends the user's source code, selected language, and problem ID to the backend	Validates the request, fetches hidden test cases, executes the submitted program	POST /api/submissions

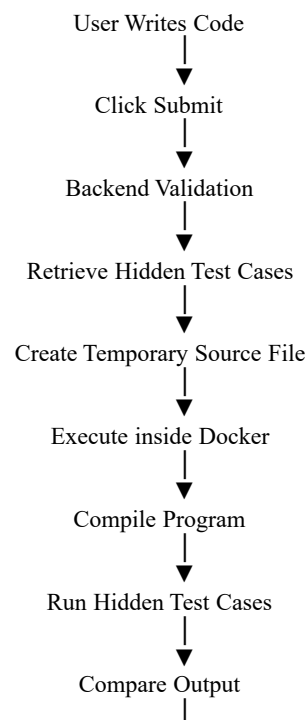
	after the user clicks the Submit button.	inside a Docker container, compares the generated output with the expected output, generates the verdict, stores execution statistics in the Submissions collection, and returns the evaluation result.	
Leaderboard	Displays the latest submissions along with verdict, execution time, and submission timestamp to monitor user performance.	Retrieves the latest ten submissions from the Submissions collection, sorts them by submission time, and returns the results to the frontend.	GET /api/leaderboard

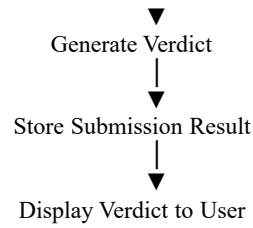
3. Code Evaluation System

- **Docker-Based Sandbox**

To ensure secure execution, every submitted program runs inside an isolated Docker container. Each container is configured with predefined CPU, memory, and execution time limits to prevent malicious programs from affecting the host machine. Once evaluation is complete, the container is automatically terminated and removed, ensuring complete isolation between different submissions.

Submission Workflow



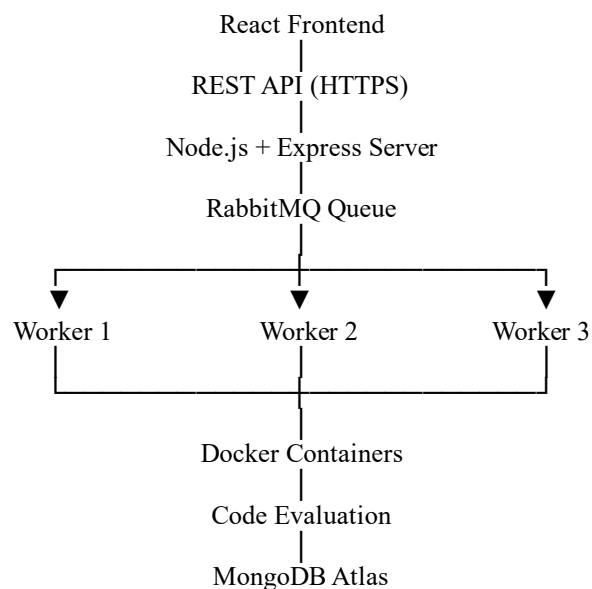


Security Matrix

Threat	Solution
Malicious Code	Docker Sandbox
Unauthorized Access	JWT
Password Theft	bcrypt
API Abuse	Rate Limiting
Infinite Loop	Time Limit
High Memory Usage	Memory Limit
SQL/NoSQL Injection	Input Validation

Deployment and Scalability

The Online Judge follows a modular architecture where the frontend, backend, and database are deployed independently, enabling easy maintenance and scalability. To support high submission volumes, the system can be extended with RabbitMQ and multiple worker services that process code submissions asynchronously, ensuring efficient load distribution and improved performance.



Future Enhancements:

To further improve scalability and performance, the evaluation system can be extended with:

- AI-assisted code review and optimization suggestions.
- Plagiarism detection using MOSS.
- RabbitMQ-based asynchronous job processing.
- Redis caching for frequently accessed problem data.
- Live Contest Mode.